

Paradigmas de Programación

[Inicio](#)[Paradigmas](#)[Manejo de HTML](#)[Manejo de CSS](#)[Formularios](#)[Tabla de Contenido](#)

1. Introducción
2. Conceptos básicos
3. Componentes de la comunicación
4. Arquitectura de 3 capas
5. Creación y configuración de un sitio web
6. Formulario

[Descargar esta pagina](#)

Conceptualización basica de paradigmas de programación

Introducción

La arquitectura cliente-servidor es la base del internet y de la gran mayoría de aplicaciones y programas que usamos hoy en día. Se basa en, como su nombre lo indica, un cliente que realiza peticiones a un servidor, esto es el pilar fundamental de todo el internet, toda página está alojada en un servidor al que tú como usuario te conectas, al igual que aplicaciones como redes sociales, tiendas de aplicaciones, tiendas virtuales, aplicaciones de banco, etc.

Que es un paradigma de programación

Un paradigma de programación es un estilo, enfoque o metodología fundamental para estructurar y resolver problemas mediante código. Define la filosofía de cómo el programador conceptualiza la lógica del software y cómo el lenguaje gestiona las acciones.

Los distintos paradigmas de programación suelen ser impuestos por cada lenguaje de programación, a su vez un lenguaje puede implementar uno o varios paradigmas al mismo tiempo. Cada uno de estos paradigmas intentan solventar las necesidades de los programadores al momento de escribir código, como puede ser, la necesidad de ordenar y modular el código o simplemente manejar un software de una manera totalmente distinta a como lo hacen otros paradigmas.

Tipos de paradigmas de programacion

Programación imperativo:

En este tipo de paradigma se detalla cada paso que el programa debe seguir para cumplir con el comportamiento esperado, permitiendo un control exacto de su flujo.

- **Programación procedural**

En este se ordena el código en procedimientos o mas conocidos como funciones, ¿Que son las funciones? Son bloques de código que podemos reutilizar en cualquier momento sin tener que rescribir todo el código. La idea es ordenar el programa en pequeñas funciones que resolveran una tarea cada una hasta que todas logren el objetivo del programa.

- **Programación orientada a objetos (OOP)**

Se basa, como su nombre lo indica, en objetos. La idea es crear clases de objetos con sus metodos (Acciones que puede realizar) y atributos (Sus características), permitiendo una modulación del código que la programación procedural nunca podría alcanzar.

¿Por que es así? por ejemplo: Al crear una clase podemos instanciar cuantos nuevos objetos de esa clase queramos en cualquier parte del código cada uno con atributos que tienen una información distinta, podemos elegir a que atributos y metodos pueden acceder otras clases, se puede heredar los metodos y atributos de una clase padre a una clase hija y tantas otras cosas que invito

investiguen por su cuenta, en verdad es muy interesante. Ejemplos de este paradigma tenemos una barbaridad Java, C++, C#, Python, JavaScript, Kotlin, etc.

Programación declarativa

Con este paradigma describimos el resultado final, no tenemos que explicar como conseguirlo la computadora determinara cual es el mejor procedimiento para ello.

- **Programación funcional**

La matriz de la programación funcional es el uso de funciones puras, estas no afectan al exterior (no modifican variables globales, no imprimen en consola, no guardan en base de datos, etc), ademas, para una entrada siempre devuelven una salida. Normalmente son lenguajes utilizados para funciones matematicas. Ejemplos: Haskell, Erlang y Lisp.

- **Programación Lógica**

En el caso de la programación logica se le entrega al sistema un conjunto de hechos, ejemplo: El agua es azul, la semana tiene 7 dias, dormir 8 horas es bueno para la salud; y unas reglas (relaciones entre estos hechos). Luego, podemos preguntarle cualquier cosa a este sistema y el mediante esta información dará una respuesta.

- **Programacion Reactiva**

Es un enfoque centrado en los flujos de datos y la propagación del cambio. En lugar que el programa pida constantemente información, se queda esperando a que la información llegue y recciona a ella.

Conceptos basicos del la arquitectura cliente-servidor

En la arquitectura cliente-servidor un cliente realiza peticiones a otro programa, el servidor, quien le da respuesta. Bajo esa simple premisa se construye por ejemplo toda la web. Este modelo destaca por la centralizacion de la información, ya que, todo se encuentra alojado en el servidor, pero, esto tiene sus matices.

Característica	Descripción
Centralizacion	Todo se encuentra alojado en un solo servidor, lo que facilita su manutención y seguridad, en contra, si el servidor cae por cualquier razon, se perdera el acceso a la información hasta que se logre levantarlo nuevamente, proceso que se hace mucho mas facil al ser un solo servidor.
Escalabilidad	Puedes añadir mas clientes facilmente o mejorar la potencia del servidor si hay mucha demanda.
Independencia	El cliente y el servidor no necesitan saber como

Característica	Descripción
	funciona el otro internamente, solo se comunican pasandose la información que necesitan.

Componentes de la comunicación entre un cliente y un servidor

Componente	Descripción
Cliente	Es el dispositivo o aplicación que inicia una petición de servicio (ejemplo: tu navegador web o una aplicación móvil).
Servidor	Es una máquina o programa potente que espera peticiones, las procesa y entrega una respuesta (ejemplo: los servidores de Google o Amazon).
Solicitud (Request)	El mensaje que envía el cliente solicitando algo (una web, un archivo, un dato).

Componente	Descripción
Respuesta (Response)	El mensaje que devuelve el servidor con la información solicitada o un error.
Mensaje	Es la información que viaja por la red, esta dividido en: metadatos (Es información que no es visible en el mensaje como quien lo envia o a quien va dirigido), el contenido del mensaje y el trailer o footer que comprueba si el mensaje llego correctamente.
Protocolo	Es el componente mas critico a nivel logico, son las reglas que establecen como se empaqueta, envian y reciben los datos. Existen muchisimos protocolos para cada tipo de conexion distinta, ejemplo: TCP/IP, HTTP/HTTPS, FTP, SSH, etc.

Arquitectura de 3 capas

La arquitectura de tres capas es una evolución del modelo cliente-servidor que busca resolver problemas de escalabilidad y mantenimiento. Su principio fundamental es la separacion de responsabilidades, dividiendo la aplicación en tres niveles logicos y, a menudo fisicos distintos.

- **La Capa de presentación:**

Es la interfaz que ve el usuario desde su navegador, la aplicación o terminal, los programadores en esta capa solo deben encargarse de la interfaz con herramientas como HTML, CSS, JavaScript, etc.

- **La Capa de Negocio**

Actúa como el núcleo o "cerebro" del sistema, situándose entre la presentación y los datos. Aquí es donde se definen todas las reglas, algoritmos y validaciones que hacen que la aplicación funcione según lo planeado. Recibe las peticiones del usuario, procesa la información aplicando la lógica necesaria y decide qué datos deben ser consultados o guardados, funcionando como el motor que mueve toda la operación.

- **La Capa de Datos**

Es la encargada de la persistencia y gestión de la información en el sistema. Su responsabilidad exclusiva es almacenar, recuperar y actualizar los datos en una base de datos o sistema de archivos de forma segura y eficiente. Esta capa no conoce las reglas del negocio ni cómo se ve la interfaz; simplemente responde a las órdenes de la capa de negocio para entregar o guardar la información solicitada de manera organizada.

Creación y configuración de un sitio web:

La creación de un sitio web comienza con el entendimiento de que este es, en esencia, un conjunto de archivos digitales alojados en una

computadora conectada permanentemente a la red, llamada servidor. Para que este contenido sea accesible, se requiere de un Hosting (alojamiento), que es el espacio físico o virtual donde residen los datos, y un dominio, que funciona como la dirección legible (ej. google.com) que apunta a la dirección IP técnica de dicho servidor.

La configuración inicial implica vincular estos dos elementos mediante los DNS (Sistema de Nombres de Dominio), que actúan como un directorio que traduce el nombre del dominio en la ubicación exacta del servidor. Una vez establecida la conexión, el servidor se encarga de procesar las peticiones de los usuarios y enviar los archivos correspondientes al navegador del cliente, donde se interpretan para mostrar la interfaz visual.

A continuación un video tutorial de [Programación Accesible](#) sobre como crear una pagina web con Github pages al igual que este blog:

**Evaluación: Arquitectura
Cliente-Servidor**

1. En la arquitectura cliente-servidor, el cliente siempre inicia la comunicación.

☐ Verdadero ☐ Falso

2. El servidor no necesita estar conectado a internet para funcionar.

☐ Verdadero ☐ Falso

3. La arquitectura de 3 capas incluye: Presentación, Negocio y Datos.

☐ Verdadero ☐ Falso

4. Los DNS traducen direcciones IP a nombres de dominio legibles.

☐ Verdadero ☐ Falso

5. El protocolo HTTP es un ejemplo de protocolo de comunicación.

☐ Verdadero ☐ Falso

[<-Regresar al inicio](#)